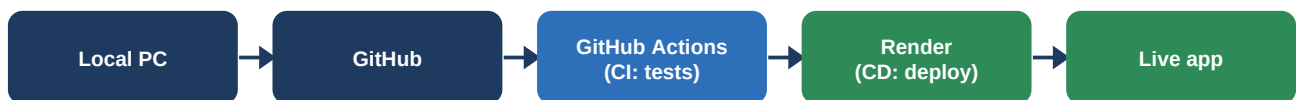


CI/CD Pipeline with GitHub Actions

A beginner-friendly guide, with the exact code and commands used to build a working pipeline for a Flask app



Project	ci_cd_demo (Flask web app)
Repository	github.com/1313rupinder/ci_cd_demo
Tools	Git, GitHub, GitHub Actions, Python 3.12, pytest, Flask, Gunicorn, Render
Prepared by	Rupinder Singh

1. What is CI/CD?

CI/CD is a way of shipping software where tests and deployment run automatically every time code changes, instead of a person doing each step by hand.

Term	Meaning	In this project
CI Continuous Integration	Every push is built and tested automatically, so a broken change is caught within seconds.	GitHub Actions installs the packages and runs pytest on every push and pull request.
CD Continuous Delivery / Deployment	Code that passes the tests is shipped to a server automatically.	Render builds and runs the app and keeps it live.
Pipeline	The full chain of automatic steps from push to live app.	push -> test -> deploy

Why it matters

- **Fast feedback:** a failing test shows up as a red cross within seconds of the push.
- **Consistency:** tests run on a clean machine every time, so "it works on my PC" stops being an excuse.
- **Less manual work:** no copying files to a server by hand.
- **Safer releases:** a change that fails its tests can be stopped before it reaches users.

Can GitHub do the whole pipeline?

Yes. **GitHub Actions** is GitHub's built-in automation. A *workflow file* inside the repository tells GitHub what to run and when. In this project it runs the CI (tests). The one thing GitHub does not do is host a running web app for the public. A separate host such as Render runs the app and deploys each new version from your GitHub repository.

Part	Who does it	How
Store the code	GitHub	Git repository
CI (test)	GitHub Actions	<code>.github/workflows/ci.yml</code>
CD (deploy)	Render	Auto-Deploy on every new commit (Section 8)
Run the app for users	A host such as Render	Gunicorn web server

2. What you need before starting

- A free **GitHub** account and a new repository (this project uses `ci_cd_demo`).
- **Git** installed on your computer, and **Python 3.12** (or close to it).
- A free **Render** account (render.com), connected to your GitHub.
- A terminal. On Windows use Git Bash so the commands below work as written.

Check your tools:

```
git --version
python --version
```

If python fails on Windows, try `py --version` and use `py` in place of `python` everywhere.

3. Project files

The finished project has this layout:

```
ci_cd_demo/
|-- .github/
|   |-- workflows/
|       |-- ci.yml          <- the pipeline (CI)
|-- app.py                  <- Flask app + two small functions
|-- test_app.py             <- automated tests
|-- requirements.txt         <- packages to install
```

Step 1: name the branch main

Inside your project folder, run:

```
git branch -M main
```

`git branch -M main` renames your branch to `main`, which is GitHub's default. Older Git installs call it `master`, and a mismatch breaks the workflow trigger later.

4. The application and its tests

A pipeline needs something to test. This app has a home page and two small functions.

requirements.txt

```
pytest
flask
gunicorn
```

`pytest` runs the tests, `flask` builds the web app, and `gunicorn` is the production web server `Render` uses. Flask's built-in server is for development only.

app.py

```
import os
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "CI/CD demo is running"

def add(a, b):
    return a + b

def is_even(n):
    return n % 2 == 0

if __name__ == "__main__":
    port = int(os.environ.get("PORT", 8000))
    app.run(host="0.0.0.0", port=port)
```

- `@app.route("/")` says: when someone opens the main address, run `home()`.

- `os.environ.get("PORT", 8000)` reads the port that the host assigns (Render sets PORT) and falls back to 8000 on your PC.
- `host="0.0.0.0"` lets the app accept outside connections, which a server needs.
- The `if __name__ == "__main__"` block only runs when you start the file directly, not when the tests import it.

test_app.py

```
from app import app, add, is_even

def test_add():
    assert add(2, 3) == 5

def test_is_even():
    assert is_even(4) is True
    assert is_even(7) is False

def test_home():
    client = app.test_client()
    response = client.get("/")
    assert response.status_code == 200
    assert b"running" in response.data
```

pytest runs every function whose name starts with `test_`. An `assert` line means "I claim this is true"; if it is false, the test fails. `app.test_client()` is a fake browser inside Python, so no real server is needed.

Run the tests on your own machine first

```
python -m pip install -r requirements.txt
python -m pytest
```

Expected result: **3 passed**. Always check locally first so you do not push broken code to the pipeline.

5. Building CI with GitHub Actions

GitHub looks for workflow files in `.github/workflows/`. Any `.yaml` file there becomes a pipeline.

Step 2: create the workflow file

```
mkdir -p .github/workflows
cat > .github/workflows/ci.yml << 'EOF'
name: CI

on:
  push:
    branches: [main]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Get the code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"

      - name: Install packages
        run: pip install -r requirements.txt

      - name: Run tests
        run: pytest
EOF
```

What each part does

Line	Meaning
name: CI	The label shown in the Actions tab.
on: push / pull_request	The triggers. Run on every push to main and on every pull request.
jobs: test:	A job is a group of steps that runs on its own fresh machine.
runs-on: ubuntu-latest	GitHub lends you a clean Linux machine for the job.
actions/checkout@v4	Downloads your repository's code onto that machine.
actions/setup-python@v5	Installs Python 3.12.
pip install -r requirements.txt	Installs the same packages you use locally.
pytest	Runs the tests. If any test fails, the step fails and the whole run turns red.

YAML is strict about spaces. Indent with spaces, never tabs, and keep the nesting exactly as shown. Wrong indentation is the most common reason a workflow fails to start.

Step 3: commit and push

```
git add .
git commit -m "Add Flask app, tests and CI workflow"
git push
```

`git add .` stages your changes, `git commit` saves a snapshot with a message, and `git push` uploads it to GitHub, which starts the pipeline.

Step 4: watch it run

- Open your repository on GitHub and click the **Actions** tab.
- Click the newest run, then the **test** job. Open each step to read its log.
- A green tick means every step passed. A red cross means something failed.

6. Prove that CI catches mistakes

A pipeline that always says "pass" is useless. Break a test on purpose and confirm it fails.

Break a test

```
sed -i 's/== 5/== 6/' test_app.py
git diff
git add .
git commit -m "Break a test on purpose"
git push
```

`sed -i 's/== 5/== 6/'` finds `== 5` in the file and replaces it with `== 6`. The Actions tab now shows a **red cross**. Open the *Run tests* step and you will see the reason, for example `assert 5 == 6` and `Process completed with exit code 1`. An exit code of 0 means success; anything else means failure.

Fix it

```
sed -i 's/== 6/== 5/' test_app.py
git add .
git commit -m "Fix the broken test"
git push
```

The Actions list now shows **green, red, green**. That history is your proof that the pipeline detects problems.

Failure emails are not guaranteed. GitHub sends them depending on your notification settings (github.com/settings/notifications). The red cross on the Actions tab is the reliable signal.

7. Preparing the app for deployment

Before a host can run the app, make sure of two things, both already included in the code above:

- `unicorn` is listed in `requirements.txt`.
- `app.py` reads the `PORT` environment variable instead of hard-coding a port.

Run the tests again, commit, and push. Wait for the green run before moving on:

```
python -m pip install -r requirements.txt
python -m pytest
git add .
git commit -m "Prepare Flask app for deployment"
git push
```

8. Deploying with Render (CD)

Render watches your GitHub repository and deploys every new commit on its own. GitHub Actions runs the tests, and Render handles the deployment.

Step 5: create the web service

- Sign in to render.com and click **New**, then **Web Service**.
- Connect your GitHub account and choose the `ci_cd_demo` repository.
- Fill in the settings from the table below, then click **Create Web Service**.

Setting	Value
Language / Runtime	Python 3
Branch	main
Root Directory	(leave empty)
Build Command	<code>pip install -r requirements.txt</code>
Start Command	<code>gunicorn app:app</code>
Instance type	Free
Environment variables	None needed (Render sets PORT itself)

`gunicorn app:app` means: in the file `app.py` (first *app*), run the Flask object named `app` (second *app*).

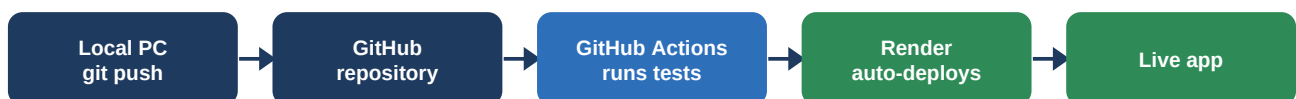
Step 6: confirm it is live

Watch the deploy logs. A successful deploy shows the packages installing, the build finishing, Gunicorn listening on `0.0.0.0:10000`, health checks returning HTTP 200, and the service marked **Live**. Open the public URL Render gives you (this project's was <https://ci-cd-demo-jrak.onrender.com>) and you will see:

CI/CD demo is running

A `favicon.ico 404` in the logs is harmless; browsers ask for a tab icon the app does not provide. On the free plan the service sleeps after inactivity, so the first request after a pause can take about a minute.

The resulting flow



Note: Render deploys as soon as it sees the new commit, even if the tests fail, because the two systems do not talk to each other.

9. Best practices

- **Test locally first** and push only when `python -m pytest` passes.
- **Keep secrets out of the repo.** Use GitHub Secrets and reference them with `${{ secrets.NAME }}`.
- **Pin versions** of actions (@v4, @v5) and of Python so runs are repeatable.
- **Write honest commit messages.** They become the names of your runs in the Actions tab.
- **Use pull requests** for bigger changes so CI checks them before they reach main.
- **Read the logs.** The first red line in a failed step usually names the real problem.

10. Git commands used in this guide

Command	What it does
<code>git branch -M main</code>	Renames the current branch to main.
<code>git status</code>	Shows which files changed.
<code>git diff</code>	Shows exactly what changed since the last commit.
<code>git add .</code>	Stages all changes for the next commit.
<code>git commit -m "message"</code>	Saves a snapshot with a description.
<code>git push</code>	Uploads commits to GitHub and starts the pipeline.

On Windows, messages such as "LF will be replaced by CRLF" are line-ending warnings, not errors. They are safe to ignore.

11. Quick recap

- **CI:** a workflow file in `.github/workflows/` runs your tests on every push.
- **CD:** the host (Render) deploys by itself when GitHub receives a new commit.
- **GitHub stores the code and runs the tests**, while a separate host runs your live app.